UT1

Práctica 2. Código seguro

Del dato al fallo: entrada → sink · pruebas · fuzzing

Hoja: `PPS-P2-codigo-seguro-alumnado.pdf` · Material: `PPS-P2-codigo-seguro-LAB.zip`

Lo de hoy

Bloque	Qué	Entregable
0	Montar el entorno de pruebas	—
1	Dónde vive el riesgo según el lenguaje	—
2	Seguir el dato: **entrada → sink**	**PPS-E1**
3	Cobertura y **test de regresión**	**PPS-E2**
4	**Fuzzing**: lo que tus tests no ven	PPS-E2

Trabajo **individual**. Al final, todo en tu repo.

YA · paso 0

Entorno de pruebas

```
cd pps-arranque
unzip ~/Descargas/PPS-P2-codigo-seguro-LAB.zip -d .
python3 -m venv .venv
. .venv/bin/activate
pip install pytest coverage
grep -qx '.venv/' .gitignore || echo '.venv/' >> .gitignore
```

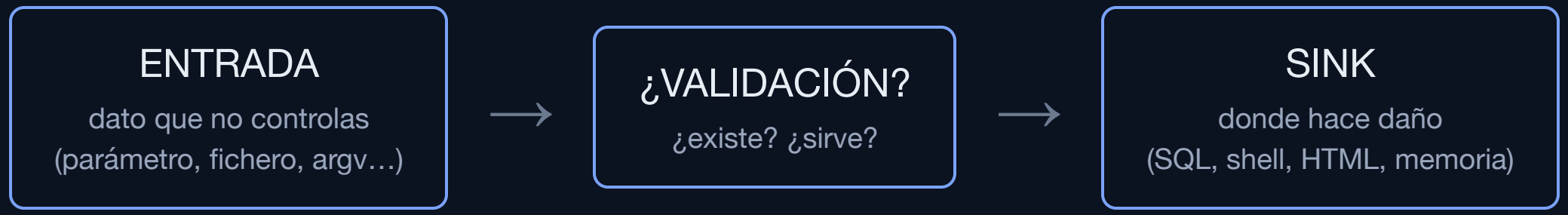
Si te sale externally-managed-environment, no estás dentro del `.venv`.

¿Dónde vive el riesgo?

Modelo	Ejemplos	Qué despliegas	Riesgo típico
Nativo	C, C++, Rust, Go	Un binario	Memoria (C/C++): desbordamientos
Interpretado	Python, PHP, JS	El **código fuente**	Código y secretos en el servidor · `eval`
Máquina virtual	Java, C#	Bytecode	Se **decompila** en segundos (jadx)

> El lenguaje decide **qué te pueden robar y cómo**.

La idea de todo el módulo



> Si un dato de fuera llega a un sink **sin validar**, hay vulnerabilidad.

El mismo dato, tres finales

```
nombre = request.args["nombre"]                                  # ENTRADA

db.execute("SELECT * FROM u WHERE n='" + nombre + "'")  # SINK SQL   → SQLi
os.system("echo " + nombre)                                      # SINK shell → inyección de comandos
render("<p>Hola " + nombre + "</p>")                             # SINK HTML  → XSS
```

La solución **no** es "limpiar" el dato: es **separar dato y código** (consulta parametrizada, nada de shell, codificar la salida).

PPS-E1 · individual

Entrada → sink

```
cd ut1/validador      # validar.py  Validar.java  validar.c
python3 validar.py ana
python3 validar.py "' OR '1'='1"      # ¿qué ha pasado?
```

En **cada** fichero: entrada · sink(s) · validación · **dónde falla según su lenguaje**.
Lo apuntas en `PPS-E1-lectura.md`, con cómo lo corregirías.

Tipos de prueba

Tipo	Pregunta que responde	Herramienta
Unitaria	¿Esta función hace lo que debe?	`pytest`
Integración	¿Las piezas encajan?	`pytest`
Regresión	¿El fallo que arreglé ha vuelto?	La suite en CI
Cobertura	¿Qué parte del código ejecutan mis tests?	`coverage`
Fuzzing	¿Qué entrada rara rompe mi código?	`atheris`, o a mano

Incidencia #17

> "Un cliente aplicó un cupón del 150 % y la tienda le devolvió un precio **negativo**."

```
cd ut1/miniproyecto
coverage run -m pytest -q
coverage report -m
```

Todo en verde. Mucha cobertura. **¿Algún test detecta el fallo?**

La cobertura mide lo que ejecutas, no lo que compruebas

```
def test_ejecuta_sin_reventar():
    aplicar_descuento(100, 10)
    assert True          # ¿qué comprueba esto?
```

- Un test que **ejecuta** código suma cobertura aunque no **compruebe** nada.
- Hacen falta aserciones que **signifiquen algo**.
- Las líneas de `Missing` son código que nadie prueba.

PPS-E2

Test de regresión: rojo → verde

1. Escribe `test_regresion_incidencia_17`: 150 % y –5 % tienen que lanzar `ValueError`.
2. Pásalo: **tiene que fallar**. Si sale verde, no prueba nada.
3. Arregla `aplicar_descuento`: **verde**.

> Un fallo **no está cerrado** hasta que hay un test que fallaría si alguien lo vuelve a meter.

Fuzzing: dejar que la máquina pruebe por ti

- Tus tests prueban lo que **a ti** se te ocurre.
- El fuzzing lanza **miles** de entradas absurdas: negativos, enormes, `inf`, `nan`…
- `ValueError` = rechazo correcto. Cualquier otra excepción o un resultado absurdo = **fallo**.
- Lo difícil no es generar entradas, es saber decir **cuándo un resultado está mal**.

PPS-E2 · paso 4

Fuzzing casero

```
python3 fuzz_casero.py
```

1. ¿Encuentra algo aunque ya arreglaste la #17?
2. Mejora `resultado_valido()`: ¿qué cumple **siempre** un precio con descuento?
3. Arregla `aplicar_descuento` y añade un test de regresión.
4. Objetivo: `0 fallos` y los tests en verde.

Para llevarse

1. El **lenguaje** decide dónde vive el riesgo.
2. Se analiza **siguiendo el dato** de la entrada al sink.
3. Cobertura alta ≠ código correcto.
4. Sin **test de regresión**, un fallo no está cerrado.
5. El **fuzzing** encuentra lo que no se te ocurrió probar.

Entrega: `ut1/` en tu repo (sin `.venv/`) + ficha de la hoja.